Experiment 'p_5_9_f_cone_surface' Results

June 4, 2026

Experiment outcome: FAILURE_VERIFY
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 4
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 4

Problem Specification

Problem name: p_5_9_f_cone_surface
Natural language statement: Write a method that computes the surface area of a cone with a circular base with a residual error less than 0.01, given radius and height.
Method signature: p_5_9_f_cone_surface(r: real, h: real) returns (surface: real)

Ensures

- `-0.01 < (((surface / (3.14159 * r)) - r) * ((surface / (3.14159 * r)) - r)) - ((h * h) + (r * r)) < 0.01`

Requires

- `r > 0.0`

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method that computes the surface area of a cone with a circular base with a
    ↪ residual error less than 0.01, given radius and height.

The signature should be:

p_5_9_f_cone_surface(r: real, h: real) returns (surface: real)

The method should respect the following contract:

requires r > 0.0, ensures -0.01 < (((surface / (3.14159 * r)) - r) * ((surface / (3.14159
    ↪  * r)) - r)) - ((h * h) + (r * r)) < 0.01

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-Dn4aC3lYb4JcqqAJI8JyVT9eAtrXW
Model: gpt-5.5-2026-04-23
Created at: 1780588060
Finish reason: stop
Usage: 406 tokens in, and 6060 tokens out

```
{
  assert 3.14159 > 0.0;
  assert 3.14159 * r > 0.0;
  assert h * h >= 0.0;
  assert r * r > 0.0;
  assert h * h + r * r > 0.0;

  var root: real :| root * root == h * h + r * r;
  var slant := if root >= 0.0 then root else -root;
  assert slant * slant == h * h + r * r;

  surface := 3.14159 * r * (r + slant);

  assert 3.14159 * r != 0.0;
  assert surface / (3.14159 * r) == r + slant;
  assert (surface / (3.14159 * r)) - r == slant;
  assert ((surface / (3.14159 * r)) - r) * ((surface / (3.14159 * r)) - r) == h * h + r *
      ↪  r;
```

```
  assert (((surface / (3.14159 * r)) - r) * ((surface / (3.14159 * r)) - r)) - ((h * h) +
      ↪  (r * r)) == 0.0;
  assert -0.01 < (((surface / (3.14159 * r)) - r) * ((surface / (3.14159 * r)) - r)) - ((
      ↪ h * h) + (r * r)) < 0.01;
}
```

Program → GenAI

```
When using dafny verify, the below error is emitted and verify fails:

problem_data/p_5_9_f_cone_surface/program.dfy(11,17): Error: cannot establish the
    ↪ existence of LHS values that satisfy the such-that predicate. Note, no trigger was
    ↪  found for the such-that predicate, which may be the reason the proof failed. To
    ↪ give a trigger explicitly, use the {:trigger} attribute. For more information, see
    ↪  the section on quantifier instantiation rules in the reference manual.
   |
11 | var root: real :| root * root == h * h + r * r;
   | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

problem_data/p_5_9_f_cone_surface/program.dfy(18,2): Error: assertion might not hold
   |
18 | assert surface / (3.14159 * r) == r + slant;
   | ^^^^^^


Dafny program verifier finished with 1 verified, 2 errors

Correct the error in code by altering only the method body. Produce and show only the
    ↪ Dafny body, including the curly braces that surround it. Do not show the signature
    ↪  nor contract. You must not use 'assume' nor 'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-Dn4cwzGhTsatKsKC8qIhlb76Yu739
Model: gpt-5.5-2026-04-23
Created at: 1780588230
Finish reason: stop
Usage: 1110 tokens in, and 10316 tokens out

```
{
  var y := h * h + r * r;
  assert h * h >= 0.0;
  assert r * r > 0.0;
  assert y > 0.0;

  var x := if y >= 1.0 then y else 1.0;
  if y >= 1.0 {
    assert x == y;
    assert y * y >= y;
    assert x * x >= y;
```

```
  assert x > 0.0;
} else {
  assert x == 1.0;
  assert x * x == 1.0;
  assert x * x >= y;
  assert x > 0.0;
}

while x * x - y >= 0.01
  invariant y > 0.0
  invariant x > 0.0
  invariant x * x >= y
  decreases 100000.0 * (x * x - y)
{
  var oldX := x;
  var e := oldX * oldX - y;
  assert e >= 0.01;
  assert e > 0.0;
  assert oldX > 0.0;
  assert oldX * oldX > y;
  assert e < oldX * oldX;
  assert 2.0 * oldX > 0.0;

  var d := e / (2.0 * oldX);
  assert d > 0.0;
  assert 2.0 * oldX * d == e;
  assert d < oldX / 2.0;
  assert d * d < d * oldX / 2.0;
  assert d * oldX / 2.0 == e / 4.0;
  assert d * d < e / 4.0;

  x := oldX - d;
  assert x > 0.0;
  calc {
    x * x - y;
  ==
    (oldX - d) * (oldX - d) - y;
  ==
    oldX * oldX - 2.0 * oldX * d + d * d - y;
  ==
    e - 2.0 * oldX * d + d * d;
  ==
    d * d;
  }
  assert x * x - y == d * d;
  assert x * x >= y;
  assert x * x - y < e / 4.0;
  assert 100000.0 * (x * x - y) < 25000.0 * e;
  assert 25000.0 * e <= 100000.0 * e - 1.0;
  assert 100000.0 * (x * x - y) < 100000.0 * e - 1.0;
}
```

```
  assert x * x - y >= 0.0;
  assert x * x - y < 0.01;

  var denom := 3.14159 * r;
  assert 3.14159 > 0.0;
  assert denom > 0.0;
  assert denom != 0.0;

  surface := (r + x) * denom;
  assert surface == (r + x) * denom;
  assert surface == denom * (r + x);
  assert surface / denom == r + x;
  assert denom == 3.14159 * r;
  assert surface / (3.14159 * r) == r + x;
  assert (surface / (3.14159 * r)) - r == x;
  assert ((surface / (3.14159 * r)) - r) * ((surface / (3.14159 * r)) - r) == x * x;
  assert (((surface / (3.14159 * r)) - r) * ((surface / (3.14159 * r)) - r)) - ((h * h) +
      ↪  (r * r)) == x * x - y;
  assert -0.01 < x * x - y;
  assert x * x - y < 0.01;
}
```

Program → GenAI

```
When using dafny verify, the below error is emitted and verify fails:

problem_data/p_5_9_f_cone_surface/program.dfy(8,2): Error: assertion might not hold
  |
8 | assert y > 0.0;
  | ^^^^^^


Dafny program verifier finished with 1 verified, 1 error

Correct the error in code by altering only the method body. Produce and show only the
    ↪ Dafny body, including the curly braces that surround it. Do not show the signature
    ↪  nor contract. You must not use 'assume' nor 'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-Dn4hHU6j4r5w9qPMdAURfwuvRUG7T
Model: gpt-5.5-2026-04-23
Created at: 1780588499
Finish reason: stop
Usage: 2076 tokens in, and 5483 tokens out

```
{
  var y := h * h + r * r;
  assert h * h >= 0.0;
```

```
assert r * r > 0.0;
assert h * h + r * r > 0.0;
assert y == h * h + r * r;
assert y > 0.0;

var x := if y >= 1.0 then y else 1.0;
if y >= 1.0 {
  assert x == y;
  assert y * y >= y;
  assert x * x >= y;
  assert x > 0.0;
} else {
  assert x == 1.0;
  assert x * x == 1.0;
  assert x * x >= y;
  assert x > 0.0;
}

while x * x - y >= 0.01
  invariant y > 0.0
  invariant x > 0.0
  invariant x * x >= y
  decreases 100000.0 * (x * x - y)
{
  var oldX := x;
  var e := oldX * oldX - y;
  assert e >= 0.01;
  assert e > 0.0;
  assert oldX > 0.0;
  assert oldX * oldX >= y;
  assert e == oldX * oldX - y;
  assert e < oldX * oldX;
  assert 2.0 * oldX > 0.0;

  var d := e / (2.0 * oldX);
  assert d > 0.0;
  assert 2.0 * oldX * d == e;
  assert d < oldX / 2.0;
  assert d * d < d * oldX / 2.0;
  assert d * oldX / 2.0 == e / 4.0;
  assert d * d < e / 4.0;

  x := oldX - d;
  assert x > 0.0;
  calc {
    x * x - y;
  ==
    (oldX - d) * (oldX - d) - y;
  ==
    oldX * oldX - 2.0 * oldX * d + d * d - y;
  ==
```

```
    e - 2.0 * oldX * d + d * d;
  ==
    d * d;
  }
  assert x * x - y == d * d;
  assert x * x >= y;
  assert x * x - y < e / 4.0;
  assert 100000.0 * (x * x - y) < 25000.0 * e;
  assert 25000.0 * e <= 100000.0 * e - 1.0;
  assert 100000.0 * (x * x - y) < 100000.0 * e - 1.0;
 }

 assert x * x - y >= 0.0;
 assert x * x - y < 0.01;

 var denom := 3.14159 * r;
 assert 3.14159 > 0.0;
 assert denom > 0.0;
 assert denom != 0.0;

 surface := denom * (r + x);
 assert surface == denom * (r + x);
 assert surface / denom == r + x;
 assert denom == 3.14159 * r;
 assert surface / (3.14159 * r) == r + x;
 assert (surface / (3.14159 * r)) - r == x;
 assert ((surface / (3.14159 * r)) - r) * ((surface / (3.14159 * r)) - r) == x * x;
 assert (((surface / (3.14159 * r)) - r) * ((surface / (3.14159 * r)) - r)) - ((h * h) +
     ↪  (r * r)) == x * x - y;
 assert -0.01 < x * x - y;
 assert x * x - y < 0.01;
}
```

Program → GenAI

```
When using dafny verify, the below error is emitted and verify fails:

problem_data/p_5_9_f_cone_surface/program.dfy(10,2): Error: assertion might not hold
   |
10 | assert y > 0.0;
   | ^^^^^^


Dafny program verifier finished with 1 verified, 1 error

Correct the error in code by altering only the method body. Produce and show only the
    ↪ Dafny body, including the curly braces that surround it. Do not show the signature
    ↪  nor contract. You must not use 'assume' nor 'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-Dn4jeQThukPio5AsH8GEUBUKvMe7J
Model: gpt-5.5-2026-04-23
Created at: 1780588646
Finish reason: stop
Usage: 3200 tokens in, and 2056 tokens out

```
{
  var hh := h * h;
  var rr := r * r;
  assert hh >= 0.0;
  assert rr > 0.0;
  var y := hh + rr;
  assert y >= rr;
  assert y > 0.0;
  assert y == h * h + r * r;

  var x := if y >= 1.0 then y else 1.0;
  if y >= 1.0 {
    assert x == y;
    assert y > 0.0;
    assert y * y >= y;
    assert x * x >= y;
    assert x > 0.0;
  } else {
    assert x == 1.0;
    assert y < 1.0;
    assert x * x == 1.0;
    assert x * x >= y;
    assert x > 0.0;
  }

  while x * x - y >= 0.01
    invariant y > 0.0
    invariant y == h * h + r * r
    invariant x > 0.0
    invariant x * x >= y
    decreases 100000.0 * (x * x - y)
  {
    var oldX := x;
    var e := oldX * oldX - y;
    assert e >= 0.01;
    assert e > 0.0;
    assert oldX > 0.0;
    assert oldX * oldX >= y;
    assert e == oldX * oldX - y;
    assert e < oldX * oldX;
    assert 2.0 * oldX > 0.0;

    var d := e / (2.0 * oldX);
```

```
    assert d > 0.0;
    assert 2.0 * oldX * d == e;
    assert d < oldX / 2.0;
    assert d * d < d * oldX / 2.0;
    assert d * oldX / 2.0 == e / 4.0;
    assert d * d < e / 4.0;

    x := oldX - d;
    assert x > 0.0;
    calc {
      x * x - y;
    ==
      (oldX - d) * (oldX - d) - y;
    ==
      oldX * oldX - 2.0 * oldX * d + d * d - y;
    ==
      e - 2.0 * oldX * d + d * d;
    ==
      d * d;
    }
    assert x * x - y == d * d;
    assert x * x >= y;
    assert x * x - y < e / 4.0;
    assert 100000.0 * (x * x - y) < 25000.0 * e;
    assert 25000.0 * e <= 100000.0 * e - 1.0;
    assert 100000.0 * (x * x - y) < 100000.0 * e - 1.0;
  }

  assert x * x - y >= 0.0;
  assert x * x - y < 0.01;

  var denom := 3.14159 * r;
  assert 3.14159 > 0.0;
  assert denom > 0.0;
  assert denom != 0.0;

  surface := denom * (r + x);
  assert surface == denom * (r + x);
  assert surface / denom == r + x;
  assert denom == 3.14159 * r;
  assert surface / (3.14159 * r) == r + x;
  assert (surface / (3.14159 * r)) - r == x;
  assert ((surface / (3.14159 * r)) - r) * ((surface / (3.14159 * r)) - r) == x * x;
  assert (((surface / (3.14159 * r)) - r) * ((surface / (3.14159 * r)) - r)) - ((h * h) +
      ↪  (r * r)) == x * x - y;
  assert -0.01 < x * x - y;
  assert x * x - y < 0.01;
}
```

Final Program

```
method p_5_9_f_cone_surface(r: real, h: real) returns (surface: real)
       requires r > 0.0
       ensures -0.01 < (((surface / (3.14159 * r)) - r) * ((surface / (3.14159 * r)) - r)
           ↪ ) - ((h * h) + (r * r)) < 0.01
{
  var hh := h * h;
  var rr := r * r;
  assert hh >= 0.0;
  assert rr > 0.0;
  var y := hh + rr;
  assert y >= rr;
  assert y > 0.0;
  assert y == h * h + r * r;

  var x := if y >= 1.0 then y else 1.0;
  if y >= 1.0 {
    assert x == y;
    assert y > 0.0;
    assert y * y >= y;
    assert x * x >= y;
    assert x > 0.0;
  } else {
    assert x == 1.0;
    assert y < 1.0;
    assert x * x == 1.0;
    assert x * x >= y;
    assert x > 0.0;
  }

  while x * x - y >= 0.01
    invariant y > 0.0
    invariant y == h * h + r * r
    invariant x > 0.0
    invariant x * x >= y
    decreases 100000.0 * (x * x - y)
  {
    var oldX := x;
    var e := oldX * oldX - y;
    assert e >= 0.01;
    assert e > 0.0;
    assert oldX > 0.0;
    assert oldX * oldX >= y;
    assert e == oldX * oldX - y;
    assert e < oldX * oldX;
    assert 2.0 * oldX > 0.0;

    var d := e / (2.0 * oldX);
    assert d > 0.0;
    assert 2.0 * oldX * d == e;
    assert d < oldX / 2.0;
```

```
    assert d * d < d * oldX / 2.0;
    assert d * oldX / 2.0 == e / 4.0;
    assert d * d < e / 4.0;

    x := oldX - d;
    assert x > 0.0;
    calc {
      x * x - y;
    ==
      (oldX - d) * (oldX - d) - y;
    ==
      oldX * oldX - 2.0 * oldX * d + d * d - y;
    ==
      e - 2.0 * oldX * d + d * d;
    ==
      d * d;
    }
    assert x * x - y == d * d;
    assert x * x >= y;
    assert x * x - y < e / 4.0;
    assert 100000.0 * (x * x - y) < 25000.0 * e;
    assert 25000.0 * e <= 100000.0 * e - 1.0;
    assert 100000.0 * (x * x - y) < 100000.0 * e - 1.0;
  }

  assert x * x - y >= 0.0;
  assert x * x - y < 0.01;

  var denom := 3.14159 * r;
  assert 3.14159 > 0.0;
  assert denom > 0.0;
  assert denom != 0.0;

  surface := denom * (r + x);
  assert surface == denom * (r + x);
  assert surface / denom == r + x;
  assert denom == 3.14159 * r;
  assert surface / (3.14159 * r) == r + x;
  assert (surface / (3.14159 * r)) - r == x;
  assert ((surface / (3.14159 * r)) - r) * ((surface / (3.14159 * r)) - r) == x * x;
  assert (((surface / (3.14159 * r)) - r) * ((surface / (3.14159 * r)) - r)) - ((h * h) +
      ↪  (r * r)) == x * x - y;
  assert -0.01 < x * x - y;
  assert x * x - y < 0.01;
}
```

Total Token Usage

Input tokens: 6792
Output tokens: 23915

Reasoning tokens: 20339
Sum of 'total tokens': 30707

Experiment Timings

Overall Experiment started at 1780588060230, ended at 1780588698267, lasting 638037ms (638.04 seconds)
Iteration #4 started at 1780588644735, ended at 1780588698256, lasting 53521ms (53.52 seconds)
Iteration #1 started at 1780588060230, ended at 1780588226736, lasting 166506ms (166.51 seconds)
Iteration #2 started at 1780588226741, ended at 1780588498896, lasting 272155ms (272.16 seconds)
Iteration #3 started at 1780588498896, ended at 1780588644735, lasting 145839ms (145.84 seconds)